PROJECT 02 / BUILD ALONG

Investigating Suspicious Endpoint Activity

A step-by-step SOC portfolio walkthrough

Oluwaseyi Falode

Windows 11 ARM | UTM | Sysmon | PowerShell | Sigma | MITRE ATT&CK

This build-along manual takes a Windows endpoint from a clean virtual machine to a monitored SOC investigation lab. You install and configure Sysmon, generate controlled activity, interpret process/file/network/DNS telemetry, reconstruct process lineage, build a timeline, write Sigma detections, and validate those detections against the telemetry you generated.

Training scope: All suspicious-looking activity in this document was intentionally generated in an isolated lab. The Sigma YAML was syntax-checked, and the required rule fields were manually confirmed in Sysmon telemetry. The rules were not executed in a Sigma engine or deployed in a production SIEM.

How screenshots are used: Figures are cropped from the actual Project 2 execution screenshots. Failed commands and repetitive troubleshooting are omitted from the main walkthrough unless they teach a useful lesson.

| 28 September 2026 |

Step 0A - Read this before you start

You need	Purpose
Apple Silicon Mac or similar host	Runs the Windows lab. This project used UTM on an M3 Mac.
Windows 11 ARM64 ISO	Guest operating system for the simulated endpoint.
UTM	Virtualization layer for the Windows VM.
Sysmon ARM64	Microsoft Sysinternals endpoint telemetry collector.
PowerShell	Creates controlled activity and queries Sysmon logs.
Notepad	Creates the XML and YAML configuration/rule files.
Project folder	Keep screenshots, configs, Sigma rules and the final PDF together.

Safety: Use only your isolated lab VM. The commands in this walkthrough create benign local files, run discovery utilities, perform DNS lookups and open a test TCP connection.

Learning rule: Before running a command, answer three questions: What am I trying to prove? What event should Sysmon create? Which fields will show the answer?

Part	What you will do
Steps 1-4	Build Windows 11 ARM in UTM, install Guest Tools and prepare the endpoint.
Steps 5-8	Install Sysmon, verify the service and understand the core event IDs.
Steps 9-12	Capture Event ID 1 and reconstruct PowerShell -> CMD -> Notepad process lineage.
Steps 13-17	Generate discovery activity, identify a telemetry gap, create/apply a Sysmon config and capture File Create.
Steps 18-21	Capture network/DNS telemetry and build a scoped investigation timeline.

Steps 22-24	Interpret the simulated incident and map relevant MITRE ATT&CK techniques.
Steps 25-31	Create three Sigma rules and confirm their positive telemetry fields.
Steps 32-34	Summarize coverage, write response recommendations and capture lessons learned.
Appendices	Published Sysmon config, corrected Sigma rules, troubleshooting, evidence index and references.

Terms in plain English: Process = running program. Parent process = program that launched another program. Telemetry = evidence records. Event ID = Sysmon category number. ProcessGuid = unique Sysmon process identity. Detection rule = logic that decides which telemetry is interesting.

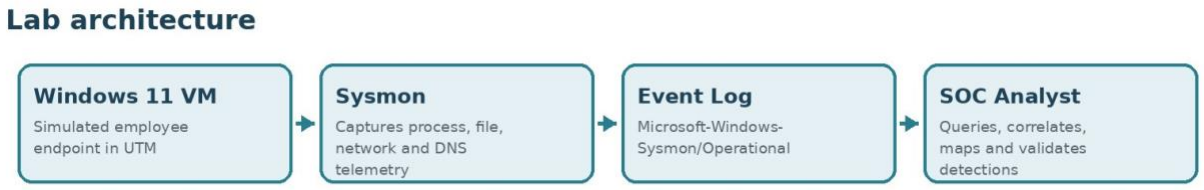
Step 0B - Your route through the lab

Figure 1. Architecture used throughout the lab.

Step 0C - Executive summary and project objective

Project 2 is a controlled endpoint-investigation and detection-engineering lab. A Windows 11 ARM endpoint was instrumented with Sysmon, then used to generate reproducible process, file, DNS and network telemetry. The investigation progressed from basic process creation to process lineage, time-scoped correlation, MITRE ATT&CK mapping and three Sigma detections.

Project question	Answer demonstrated in the lab
What ran?	Sysmon Event ID 1 exposes Image, CommandLine, User and parent fields.
What did the process create?	Event ID 11 attributes a Temp file/script to PowerShell.



Where did the endpoint communicate?	Event ID 3 exposes source/destination IP, port, protocol and initiating process.
What domain activity occurred?	Event ID 22 exposes QueryName, QueryStatus, QueryResults and process context.
Can the behavior be detected?	Three Sigma rule field sets were manually confirmed in generated Sysmon telemetry; YAML syntax was checked separately.

Outcome: Authorized simulation with no demonstrated compromise. The project demonstrates evidence interpretation, correlation and detection design rather than claiming that common administrative tools are inherently malicious.

Recruiter view: The deliverable is designed so another analyst can reproduce the lab, understand every command, compare checkpoints and evaluate the reasoning behind each conclusion.

Step 0D - Lab environment, telemetry sources and evidence scope

Component	**Role in the project**
Host	Apple Silicon Mac (M3) running UTM.
Endpoint	Windows 11 ARM virtual machine named Windows11-SOC-Lab.
Sensor	Sysmon64a / SysmonDrv.
Log channel	Microsoft-Windows-Sysmon/Operational.
Analysis shell	Windows PowerShell with Get-WinEvent.
Detection format	Sigma YAML rules stored as .yml files.
Framework	MITRE ATT&CK for supported behavior mapping.
Evidence strength	**Examples**
Direct screenshot evidence	Sysmon service running; Event IDs 1, 3, 11, 22; config update; timeline; Sigma files; three validation results.
Contextual exercise evidence	Earlier Notepad/CMD lineage and ipconfig workflow used to teach process ancestry.

Not claimed	Production SIEM deployment, enterprise EDR containment, malware execution, persistence, credential theft or exfiltration.

Evidence rule: Event timestamps and event fields take precedence over screenshot order. A screenshot appearing later in the walkthrough does not prove the underlying event occurred later.

Step 1 - Create the Windows 11 ARM virtual machine

The endpoint under investigation is a Windows 11 ARM virtual machine. On Apple Silicon, use the ARM64 Windows image rather than an x64 image.

1. Open UTM and choose Create a New Virtual Machine -> Virtualize -> Windows.
2. Select the Windows 11 ARM64 ISO. Enable the Windows/driver options offered by UTM.
3. Allocate approximately 4 GB RAM, 4 CPU cores and a 64 GB virtual disk for the lab.
4. Name the VM Windows11-SOC-Lab and start it.

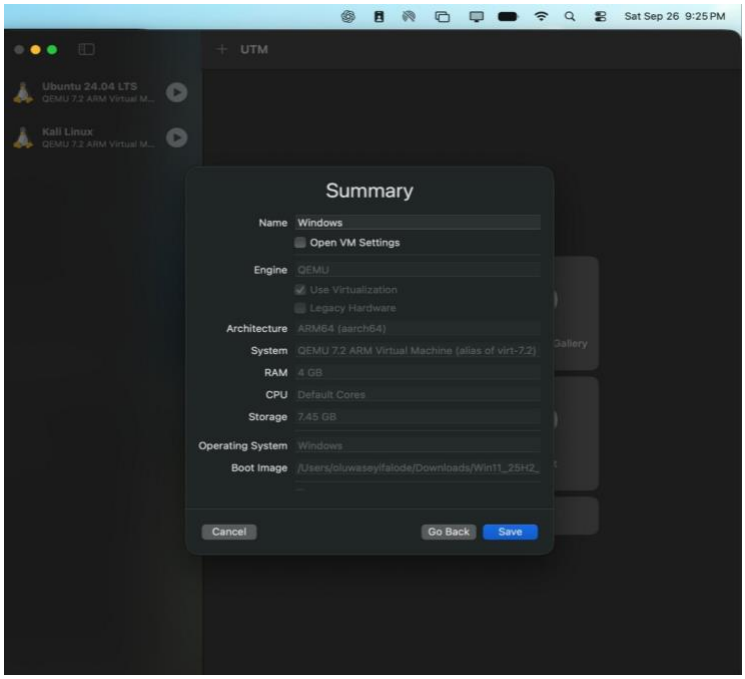


Figure 2. UTM VM summary before saving the Windows lab.

Why this matters: The VM gives you a disposable endpoint. You can generate telemetry and make configuration changes without touching a production workstation.

Checkpoint: UTM shows an ARM64 Windows virtual machine with a virtual disk and the Windows ARM ISO attached.

Step 2 - Install Windows 11

Windows Setup writes the operating system onto the virtual disk. The virtual disk is separate from your Mac storage, even though UTM stores the VM as files on the host.

5. Boot from the Windows ISO when prompted.
6. If asked for a product key, choose I do not have a product key.
7. Select Windows 11 Pro when available for the broader administrative/security feature set.
8. Choose the 64 GB unallocated virtual disk and continue.
9. Finish the Windows first-run setup and reach the desktop.

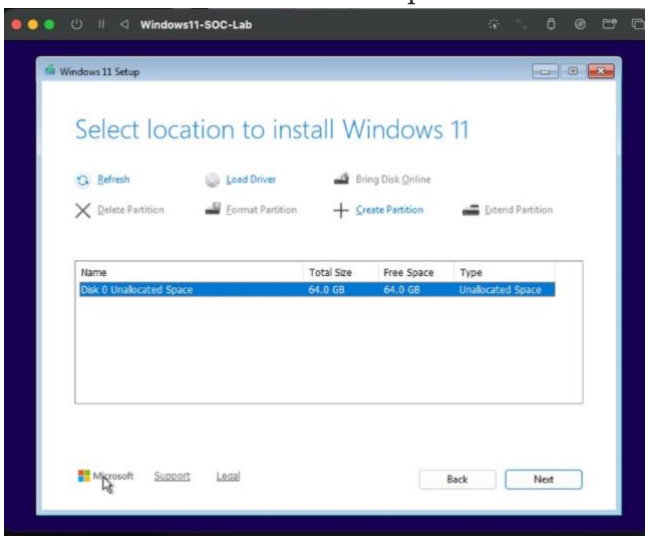


Figure 3. Windows Setup sees the 64 GB virtual disk and is ready to install.

SOC analyst lens: This is the endpoint you will monitor. Everything after this point is about instrumenting it, generating evidence and interpreting what happened.

Step 3 - Install UTM Guest Tools

Guest Tools are not a security product. They improve the connection between the Mac host and the Windows guest: display, mouse, clipboard and shared-folder support.

10. In Windows, open File Explorer -> This PC.
11. Open the UTM Guest Tools CD/DVD.
12. Run the UTM guest tools installer with default options.
13. Restart Windows after installation.

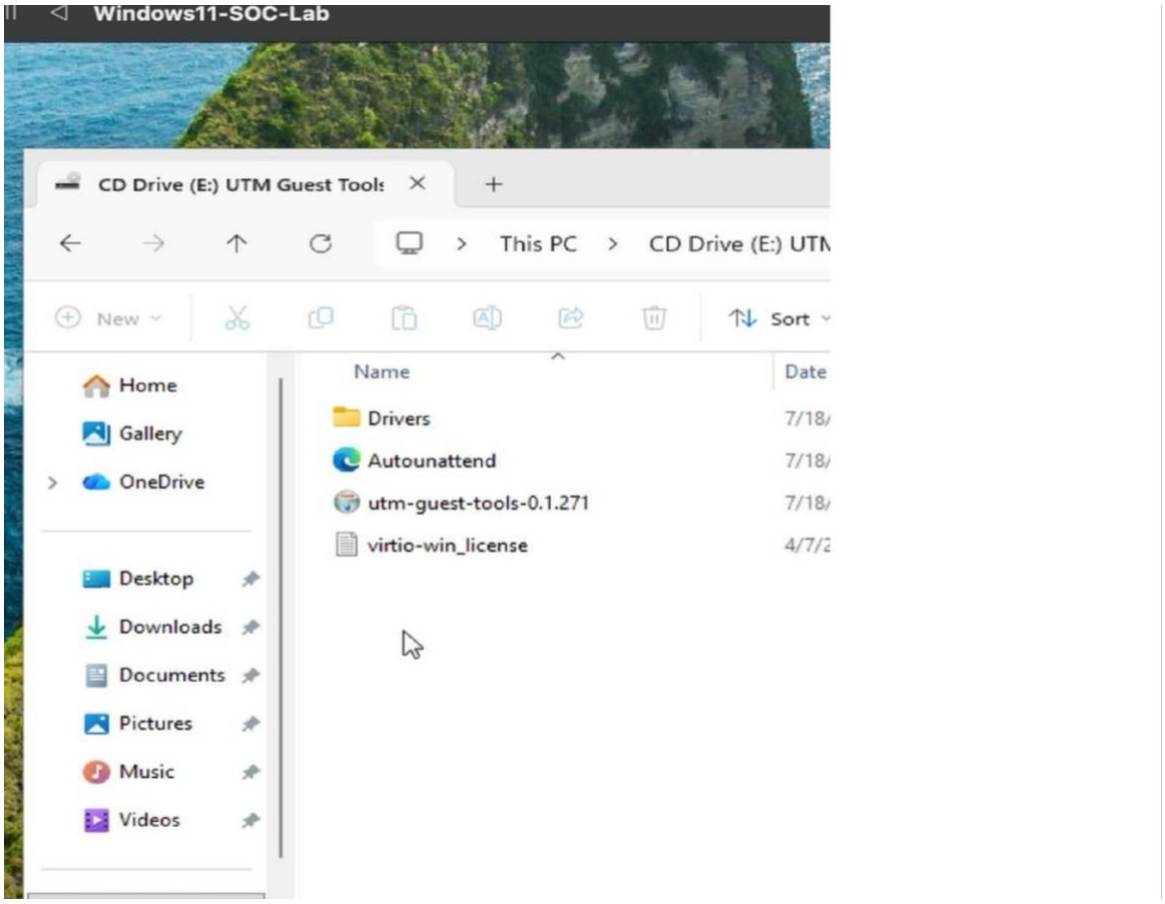


Figure 4. UTM Guest Tools media mounted inside the Windows VM.

Checkpoint: Windows restarts normally and the VM is comfortable to use for screenshots and file transfer.

Step 4 - Download and extract Sysmon

Sysmon (System Monitor) is a Microsoft Sysinternals service/driver that writes detailed endpoint activity into Windows Event Log. It becomes the sensor for this project.

14. Download Sysmon from the official Microsoft Sysinternals page.
15. Extract the archive into C:\Users\windows\Downloads\Sysmon.
16. For an ARM64 VM, use Sysmon64a.exe. **PowerShell: verify the extracted files**

```
cd "C:\Users\windows\Downloads\Sysmon"
dir
```

What to notice: Sysmon64a.exe is the ARM64 binary. Using the correct architecture matters because the VM itself is ARM64.

Step 5 - Install Sysmon

Run the installer from an elevated PowerShell window. The service will remain active in the background and write events as Windows activity occurs.

Administrator PowerShell

```
.\Sysmon64a.exe -accepteula -i
```

Command breakdown:

Part	Meaning
.\Sysmon64a.exe	Run the Sysmon ARM64 executable from the current folder.
-accepteula	Accept the Sysinternals EULA so installation can proceed.
-i	Install Sysmon as a Windows service/driver.

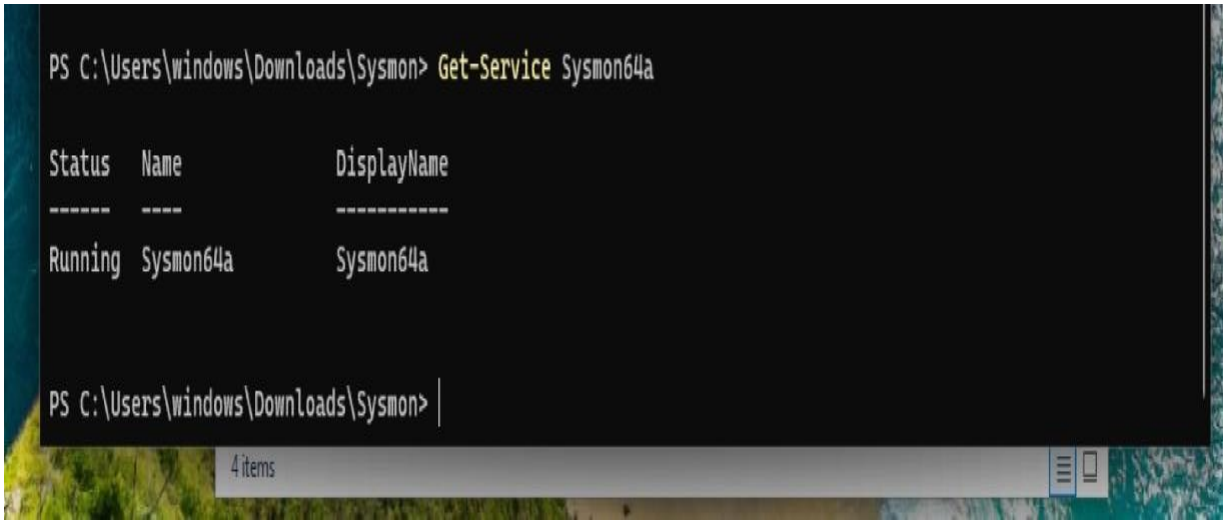


Figure 5. Sysmon installation output and service verification.

Checkpoint: Get-Service Sysmon64a returns Running. The earlier Get-Service Sysmon64 error is useful troubleshooting context, but the corrected service name is Sysmon64a.

Step 6 - Understand where Sysmon writes evidence

Sysmon writes to the Windows Event Log channel Microsoft-Windows-Sysmon/Operational. You can view it in Event Viewer or query it directly with PowerShell.

Find the Sysmon log and inspect recent events

```
Get-WinEvent -ListLog *Sysmon*
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 10
```

Event ID	Meaning	Question for the analyst
1	Process Create	What program ran? Who launched it? With what command line?
3	Network Connection	Which process communicated with which IP/port?
11	File Create	Which process created which file?
22	DNS Query	Which process looked up which domain and what did DNS return?

Important distinction: A Sysmon event is raw evidence. A detection rule is logic layered on top of evidence to decide which behavior should be reviewed.

Step 7 - Capture your first Process Create event

Start Notepad from PowerShell. Sysmon should record an Event ID 1 because a new process was created.

Generate the event

```
notepad
```

Read a recent Event ID 1

```
(Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 20 |  Where-Object {$_.Id -eq 1} |
 Select-Object -First 1).Message
```

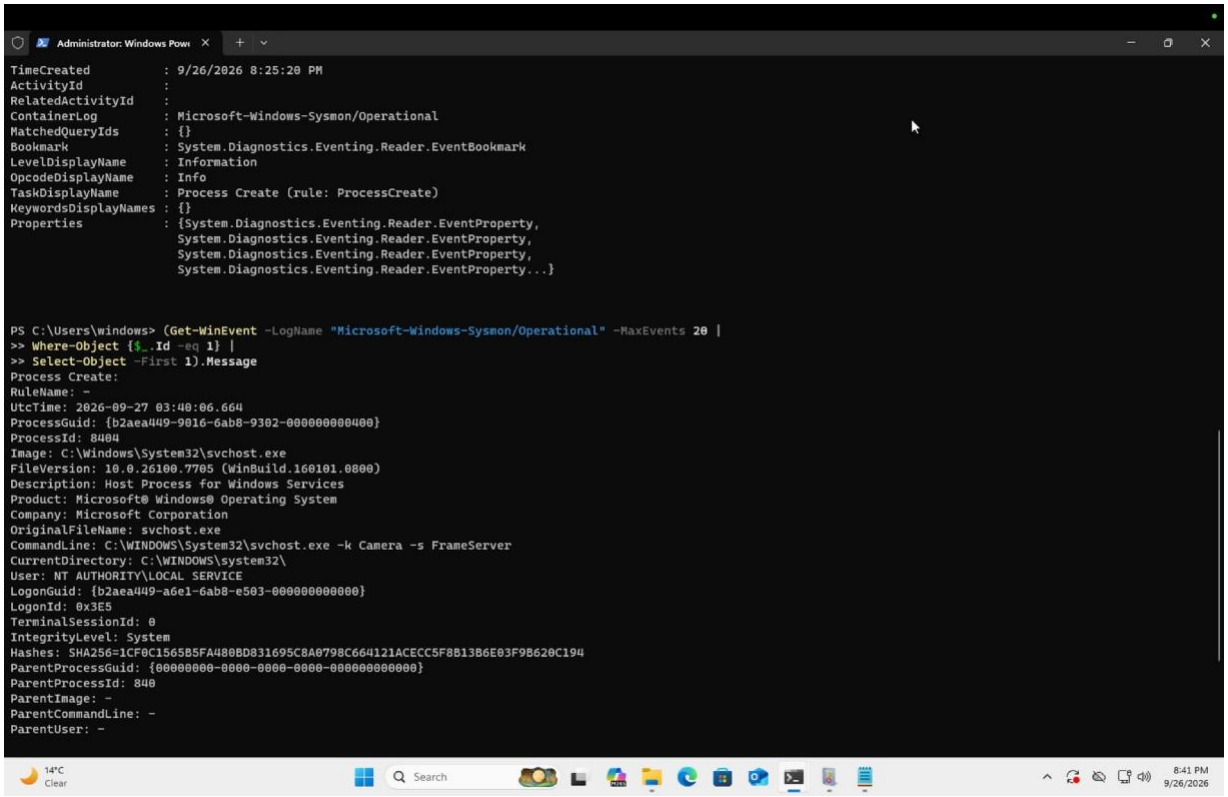


Figure 6. Sysmon Event ID 1 showing a process-create record with Image, CommandLine, User, Hashes and parent fields.

Fields to learn: Image = executable path. CommandLine = how it was launched. User = security context. Hashes = executable fingerprint. ParentImage = launcher. ParentCommandLine = how the launcher was invoked.

Step 8 - Filter noise and find the process you care about

Windows creates many processes in the background. Instead of always taking the newest event, filter the event message for the process name you generated.

Filter specifically for Notepad

```
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 100 |
Where-Object { $_.Id -eq 1 -and $_.Message -match 'notepad\.exe' } |
Select-Object -First 1 -ExpandProperty Message
```

SOC habit: Start with a time window and an indicator. Broad queries create noise; scoped queries help you prove or disprove a hypothesis.

Checkpoint: The returned event shows Notepad as Image and a parent process that explains how Notepad was launched.

Step 9 - Build a parent-child process chain

Process lineage is the family tree of execution. It explains not only what ran, but what caused it to run.

17. From PowerShell, run cmd.exe.
18. Inside Command Prompt, run notepad.exe.
19. Close Notepad and Command Prompt.
20. Query Event ID 1 for cmd.exe and notepad.exe.

```
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 100 |
Where-Object { $_.Id -eq 1 -and ($_.Message -match "cmd.exe" -or $_.Message -match "notepad.exe") } |
Select-Object -First 5 -ExpandProperty Message
```

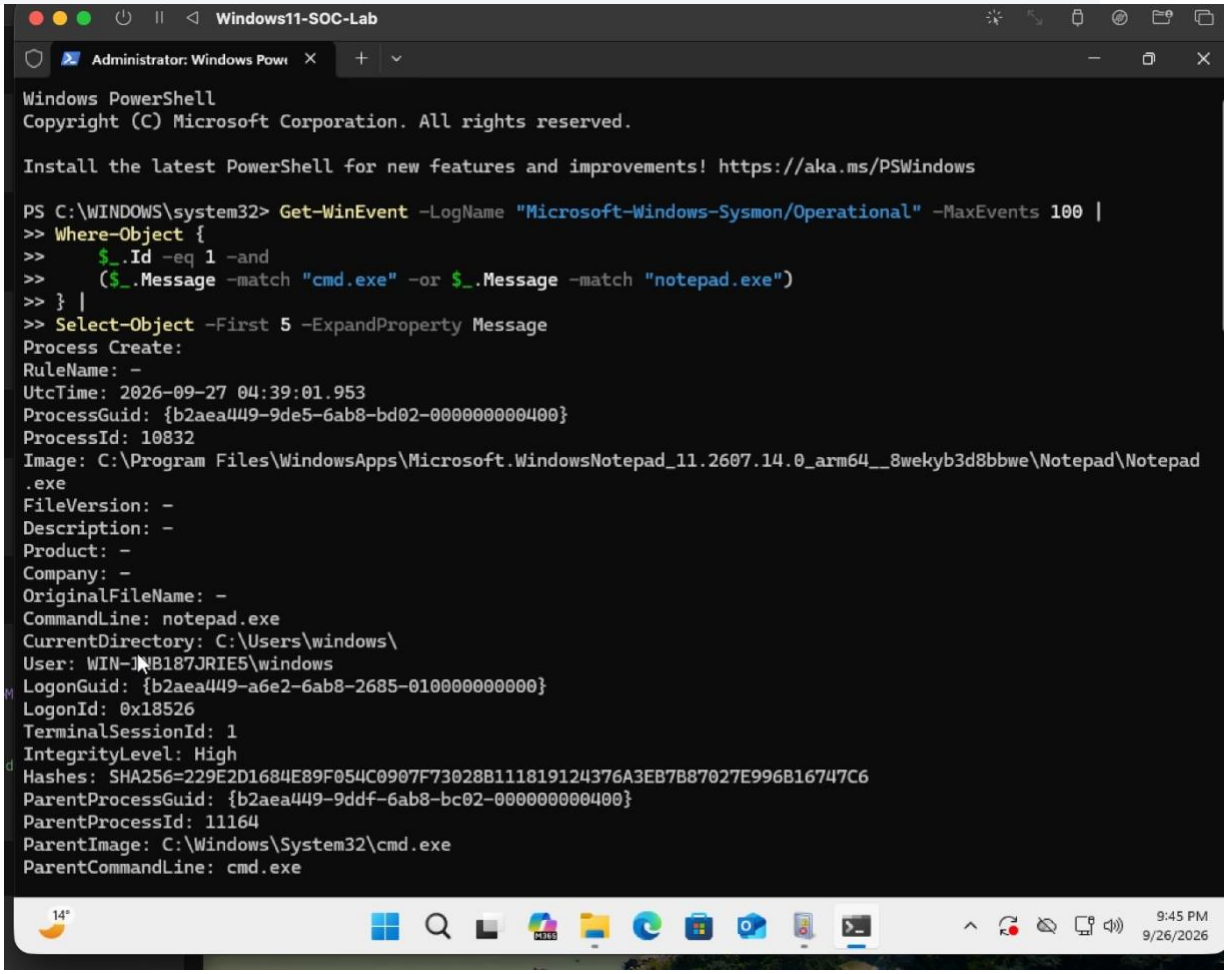


Figure 7. Notepad Event ID 1 shows cmd.exe as its parent.

What you proved: PowerShell launched CMD, then CMD launched Notepad: powershell.exe -> cmd.exe -> notepad.exe.

Step 10 - Use ProcessId and ProcessGuid to correlate events

A PID identifies a running process at one moment, but Windows can reuse PIDs. Sysmon also assigns a ProcessGuid, which is designed to uniquely identify the process instance across events.

Field	Use
ProcessId	Fast local process identifier; can be reused later.
ParentProcessId	PID of the launcher at the time the child was created.
ProcessGuid	Sysmon unique identity for one process instance.
ParentProcessGuid	Unique identity of the parent process instance.

Analyst workflow: take the child ParentProcessId/ParentProcessGuid, find the matching parent Event ID 1, then continue backward if needed.

Why this matters: Attack investigations often begin with one suspicious process. Lineage helps reconstruct the execution path that led to it.

Step 11 - Generate user and network discovery activity

Now generate two common discovery actions. They are legitimate administrative tools, but attackers also use them after execution to understand the host.

Controlled discovery activity

```
whoami
cmd.exe /c ipconfig
```

Command	Plain-English meaning	Why a SOC cares
whoami	Show the current user/security context.	Maps to user discovery and helps explain which account executed activity.
cmd.exe /c ipconfig	Start CMD, run ipconfig, then exit.	Shows local TCP/IP configuration and demonstrates multi-process lineage.

Prove ipconfig was created

```
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 100 |
Where-Object { $_.Id -eq 1 -and $_.Message -match 'ipconfig\.exe' } |
Select-Object -First 1 -ExpandProperty Message
```

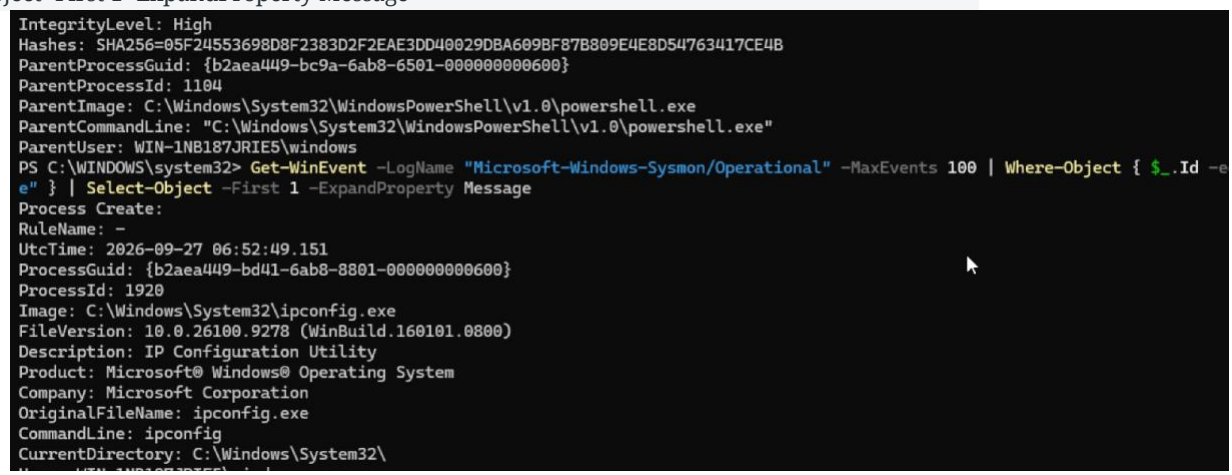


Figure 8. Clean Event ID 1 evidence showing ipconfig.exe with cmd.exe as its parent.

Step 12 - Ask the analyst questions, not just the command questions

Question	Field(s) that answer it
What ran?	Image, OriginalFileName
How was it launched?	CommandLine
Who ran it?	User, IntegrityLevel
What launched it?	ParentImage, ParentCommandLine
When?	UtcTime / TimeCreated
Can I correlate it?	ProcessGuid / ParentProcessGuid
What did it create?	Event ID 11 TargetFilename
Where did it connect?	Event ID 3 DestinationIp / DestinationPort
What domain did it ask for?	Event ID 22 QueryName / QueryResults

SOC mindset: Do not label a command malicious by itself. Ask whether the behavior makes sense for the user, parent process, host role and surrounding timeline.

Step 13 - Test file creation - and discover a telemetry gap

Create a harmless text file in the user Temp directory, then search Event ID 11 for it.

```
New-Item -Path "$env:TEMP\soc-lab-test.txt" -ItemType File -Force
Set-Content -Path "$env:TEMP\soc-lab-test.txt" -Value "Project 2 SOC Lab"

Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 200 |
Where-Object { $_.Id -eq 11 -and $_.Message -match 'soc-lab-test\.txt' } |
Select-Object -First 5 -ExpandProperty Message
```

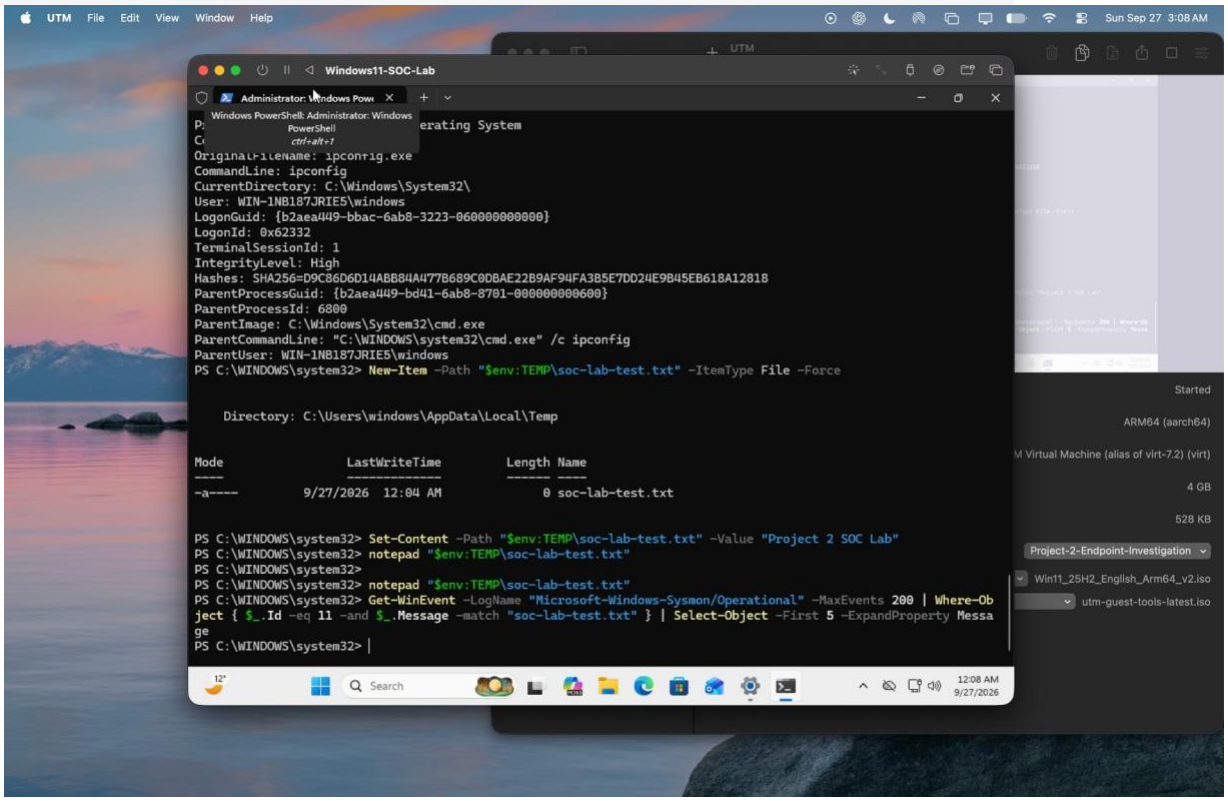


Figure 9. The file exists, but the Event ID 11 search returns no matching output.

Lesson: No result is evidence too. It means your current search/configuration did not produce the expected record. Investigate the telemetry configuration before assuming the endpoint activity was invisible.

Step 14 - Inspect the current Sysmon configuration

Show the active Sysmon configuration

```
cd "C:\Users\windows\Downloads\Sysmon"
.\Sysmon64a.exe -c
```

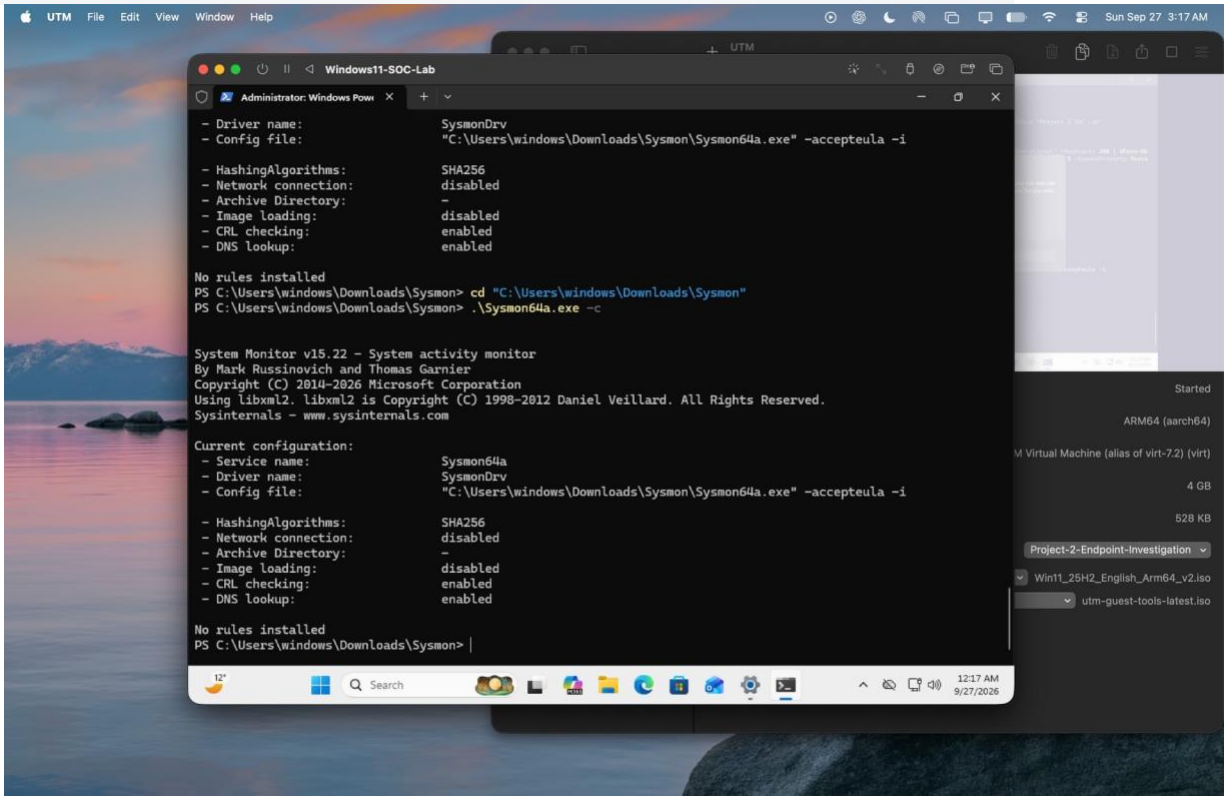


Figure 10. Initial Sysmon configuration: network disabled and no rules installed.

The important output is not that Sysmon is broken - it is running. The important point is that the initial configuration is minimal: NetworkConnect is disabled and no filtering rules are installed. Step 15 fixes that gap by explicitly enabling the process, network, file and DNS categories used in this lab.

Checkpoint: You can explain the difference between installing Sysmon and configuring what Sysmon should record.

Step 15 - Create the lab Sysmon XML configuration

A Sysmon XML file is a set of collection/filter instructions. For this lab, broad collection is acceptable because the endpoint is isolated and we want visibility for learning.

Save as C:\Users\windows\Downloads\Sysmon\sysmon-lab-config.xml

```
<Sysmon schemaversion="4.91">
  <HashAlgorithms>sha256</HashAlgorithms>
  <EventFiltering>
    <ProcessCreate onmatch="exclude"></ProcessCreate>
    <NetworkConnect onmatch="exclude"></NetworkConnect>
    <FileCreate onmatch="exclude"></FileCreate>
    <DnsQuery onmatch="exclude"></DnsQuery>
  </EventFiltering>
</Sysmon>
```

Published-config note: the original lab file used schema 4.90 and Sysmon 15.22 accepted it while reporting schema 4.91. This public version updates the example to 4.91 for consistency. Empty onmatch="exclude" sections contain no exclusions, so these categories are broadly collected in the isolated lab. This intentionally noisy approach should be tuned before production use.

Step 16 - Apply and verify the Sysmon configuration

Apply, then display the configuration

```
.\Sysmon64a.exe -c .\sysmon-lab-config.xml
.\Sysmon64a.exe -c
```

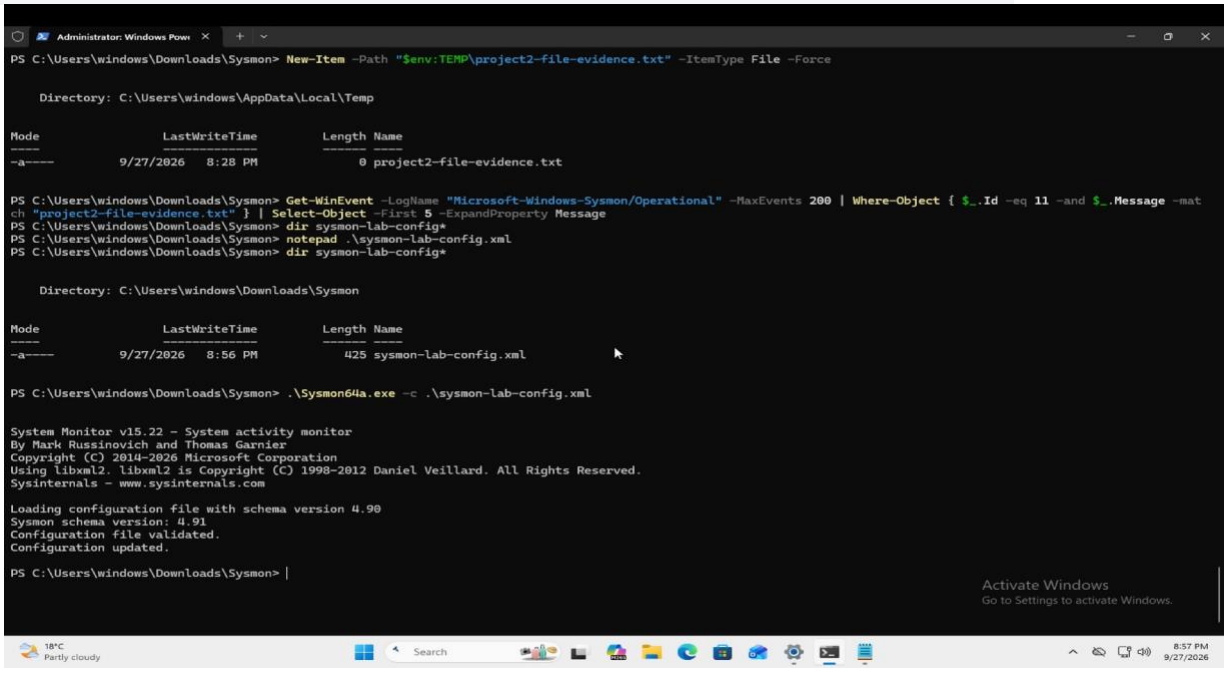


Figure 11. The XML file exists and Sysmon reports Configuration file validated / Configuration updated.

Checkpoint: Do not continue until Sysmon accepts the configuration. This prevents later missing-event troubleshooting from becoming guesswork.

Step 17 - Capture Event ID 11 - File Create

Generate and retrieve file telemetry

```
New-Item -Path "$env:TEMP\project2-file-evidence-2.txt" -ItemType File -Force
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 200 |
Where-Object { $_.Id -eq 11 -and $_.Message -match "project2-file-evidence-2.txt" } |
Select-Object -First 5 -ExpandProperty Message
```

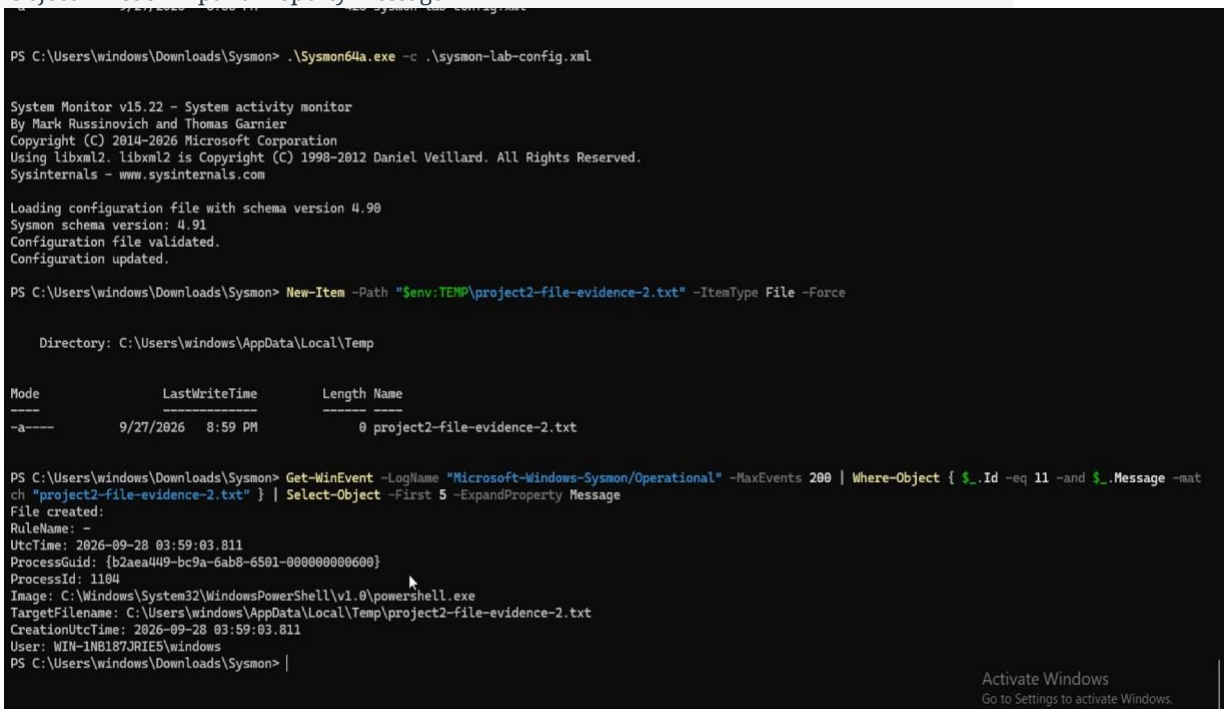


Figure 12. Event ID 11 attributes the Temp file creation to powershell.exe.

What changed from Event ID 1: Event ID 1 tells you process -> process. Event ID 11 tells you process -> file.

SOC analyst lens: File creation becomes more interesting when the path, extension and creating process are unusual - for example a script or executable appearing in a user-writable Temp directory.

Step 18 - Capture Event ID 3 - Network Connection

Event ID 3 records network connections and attributes them to a process. This answers "which program talked to which IP and port?" **Inspect recent network events**

```
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 200 | Where-Object { $_.Id -eq 3 } |
Select-Object -First 5 -ExpandProperty Message
```

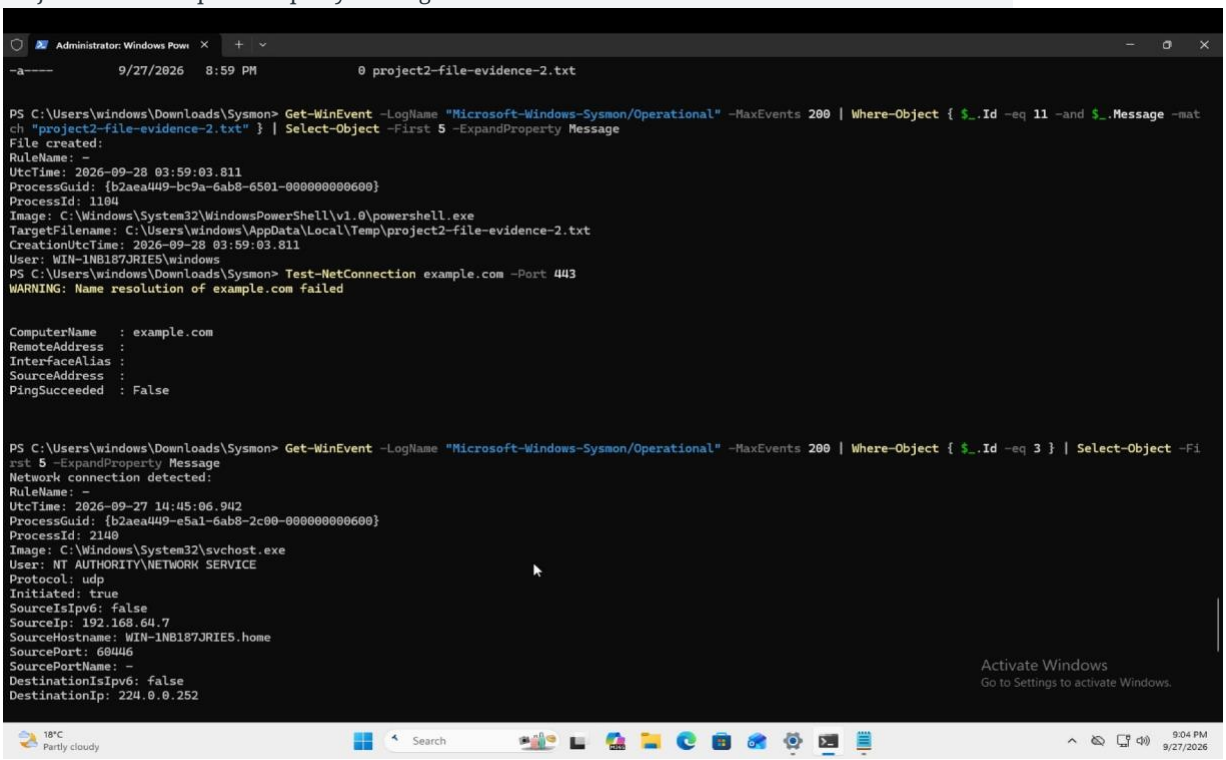


Figure 13. Sysmon Event ID 3 output exposes process, protocol, source and destination fields.

Field	Meaning
Image	Process responsible for the connection.
Protocol	TCP or UDP.
SourceIp / SourcePort	Local endpoint.
DestinationIp / DestinationPort	Remote endpoint.
Initiated	Whether the monitored process initiated the connection.

Step 19 - Capture Event ID 22 - DNS Query

DNS translates a domain name into IP addresses. nslookup is a simple way to generate a controlled DNS lookup.

Generate and inspect DNS telemetry

```
nslookup microsoft.com
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 200 |
Where-Object { $_.Id -eq 22 } |
Select-Object -First 5 -ExpandProperty Message
```

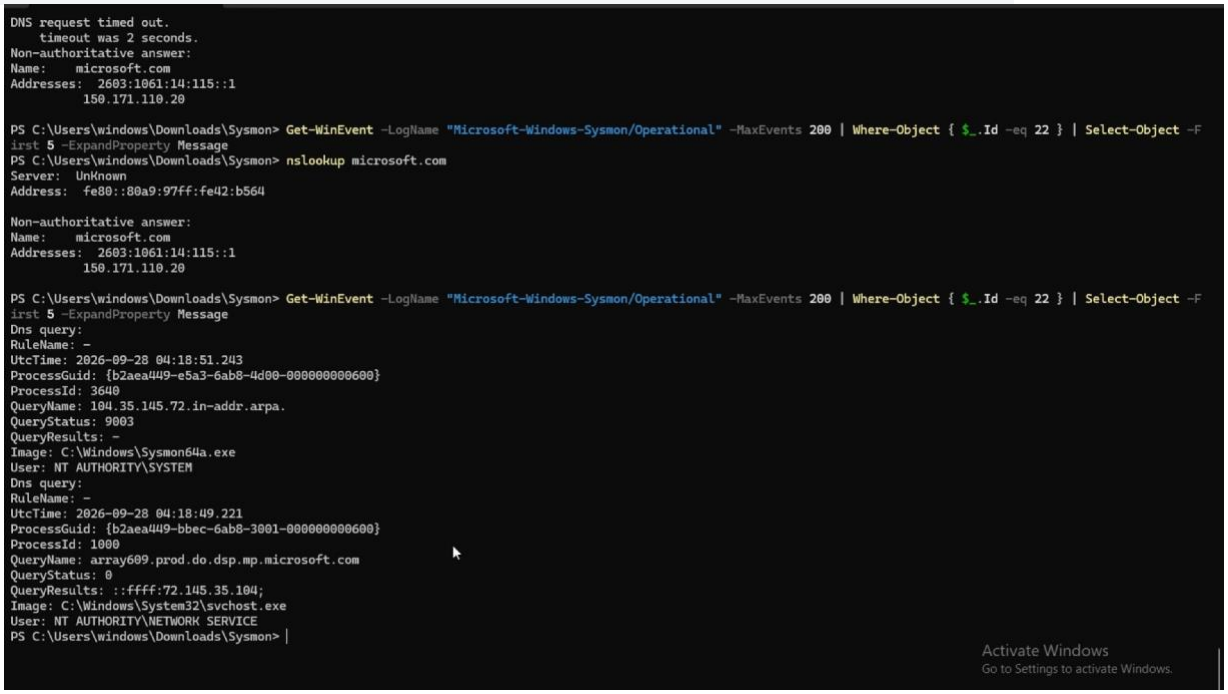


Figure 14. Event ID 22 shows process, QueryName, QueryStatus and QueryResults fields.

Plain English: The process asks "what IP belongs to this domain?" Sysmon can record the process that asked, the name requested and the result.

Important: A DNS lookup does not prove a network connection. Treat DNS and Event ID 3 as separate evidence sources and correlate them by time/process.

Step 20 - Build a chronological timeline

A timeline turns separate events into a sequence. Use a time window around the alert or activity rather than blindly searching the latest N events.

Search the last hour for the core event types

```
$start = (Get-Date).AddHours(-1)
Get-WinEvent -FilterHashtable @{
  LogName="Microsoft-Windows-Sysmon/Operational"
  StartTime=$start
Id=1,3,11,22
} | Sort-Object TimeCreated | Select-Object TimeCreated, Id, Message
```

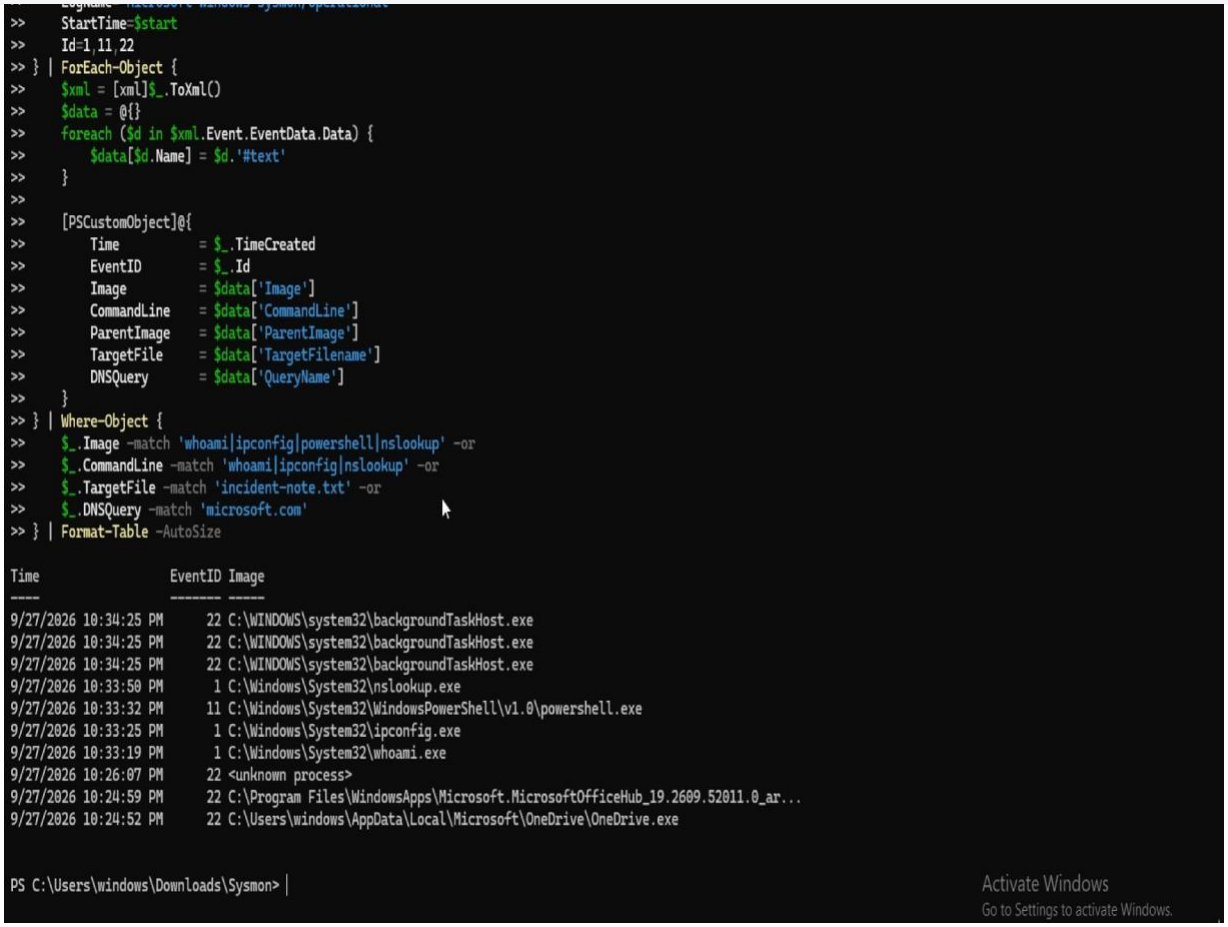


Figure 15. Process, file, network and DNS event types placed in chronological order.

Why time windows matter: In a real SOC, an alert usually has a timestamp. Analysts inspect activity before and after that point to understand context and scope.

Step 21 - Turn raw events into a readable incident table

Raw Message fields are verbose. Parse the XML fields into a small object so you can see the process, parent, file and DNS columns side by side.

Readable field extraction

```
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; StartTime=$start; Id=1,11,22} | ForEach-Object {
 $xml=[xml]$_.ToXml(); $data=@{}
 foreach($d in $xml.Event.EventData.Data){$data[$d.Name]=$d.'#text'}

[PSCustomObject]@{Time=$_.TimeCreated;EventID=$_.Id;Image=$data['Image'];CommandLine=$data['CommandLine'];ParentImage=$data['Par
entImage'];TargetFile=$data['TargetFilename'];DNSQuery=$data['QueryName']}
} | Format-Table -AutoSize
```

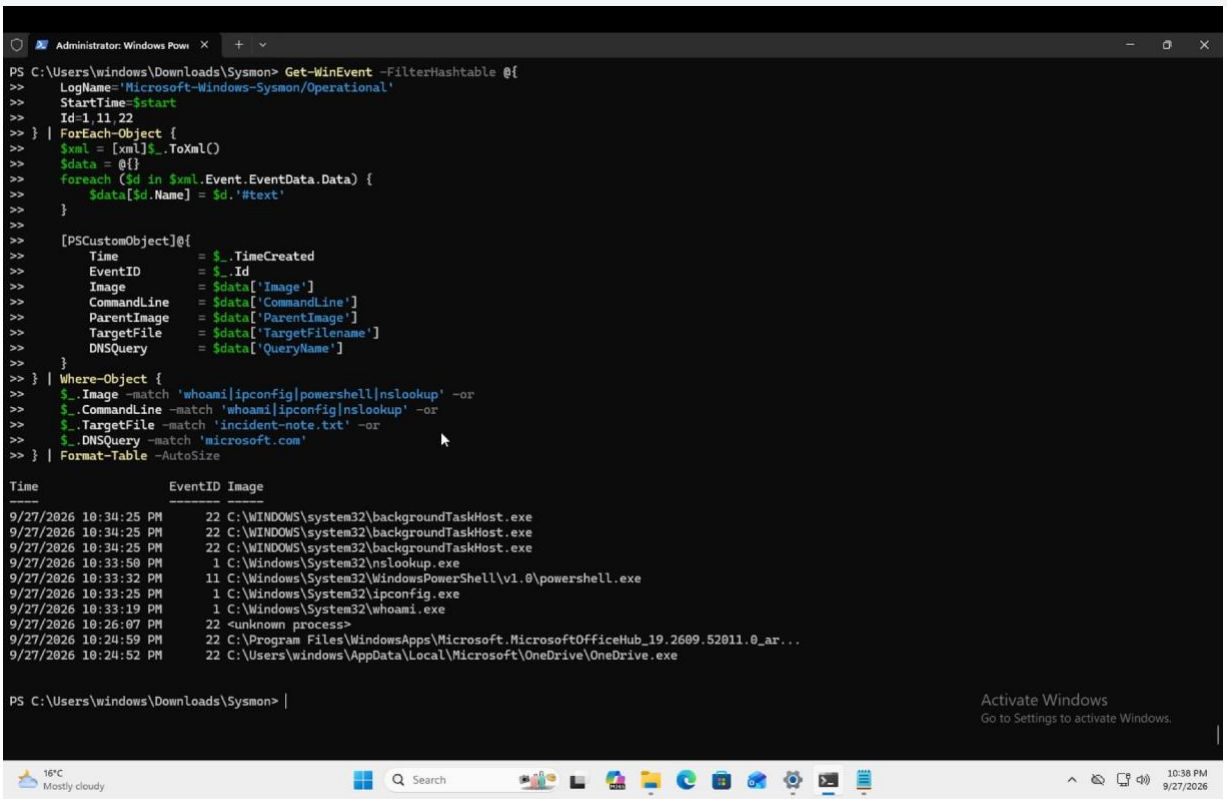


Figure 16. Filtered incident view showing relevant process, file and DNS records together.

Step 22 - Generate the final controlled incident scenario

The final scenario intentionally combines a few behaviors that are individually common but useful to correlate in a SOC investigation. **Controlled incident actions**

```
whoami
ipconfig
New-Item "$env:TEMP\incident-note.txt" -ItemType File -Force
nslookup microsoft.com
```

Behavior	What it demonstrates
whoami	User/security-context discovery.
ipconfig	Network configuration discovery.
Temp file creation	Host artifact creation attributable to PowerShell.
DNS lookup	Domain-resolution telemetry.

Scope: This is not malware and does not establish compromise. It is intentionally generated so the analyst can practice scoping, correlation and detection design.

Step 23 - Write the analyst incident story

A useful SOC narrative answers what happened, how you know, and what the evidence does not prove.

Observation	Evidence	Interpretation
whoami.exe executed	Event ID 1; PowerShell parent	User discovery behavior. Expected in this lab.
ipconfig.exe executed	Event ID 1; discovery exercise	Network configuration discovery. Can be benign administration.
PowerShell created a Temp file	Event ID 11 TargetFilename	File-system activity attributed to PowerShell.
DNS activity observed	Event ID 22 QueryName/QueryResults	Name-resolution visibility; not the same as a connection.
Separate network validation event	Event ID 3 Image/Destination fields	Confirms network telemetry and Rule 3 fields, but is not asserted as part of the exact Step 22 command sequence.

Disposition: Authorized simulation. No demonstrated compromise. The value of the exercise is the evidence workflow and the detections, not pretending that benign lab activity is malware.

Timeline caveat: Use UtcTime/TimeCreated rather than screenshot order. The Event ID 3 record used later for network-rule field confirmation has an earlier timestamp than the final validation screenshot. It is therefore treated as a separate network validation record, not as proof that Step 22 generated that event.

Step 24 - Map the behavior to MITRE ATT&CK

MITRE ATT&CK gives teams a common language for adversary behaviors. Mapping does not mean the lab activity is malicious; it describes what the behavior resembles.

Technique	Lab behavior	Why it maps
T1033 - System Owner/User Discovery	whoami.exe	Identifies the current user/security context.
T1016 - System Network Configuration Discovery	ipconfig.exe	Reveals TCP/IP and network configuration.
T1059.001 - PowerShell	PowerShell launches utilities, creates a script file and initiates a connection	Shows use of the PowerShell command/script interpreter.

Do not over-map: File creation and port 443 do not automatically justify persistence, command-and-control or exfiltration techniques. Map only what the evidence supports.

Step 25 - Understand Sigma before writing a rule

Sigma is a portable detection-rule format. Think of it as a standard way to describe "what telemetry pattern should be interesting?" A backend can later translate the logic into a SIEM-specific query.

Layer	Role
Sysmon	Generates endpoint telemetry.
Windows Event Log	Stores the telemetry.
Sigma rule	Describes the pattern to detect.
SIEM/backend	Would translate/run the rule and create alerts in production.

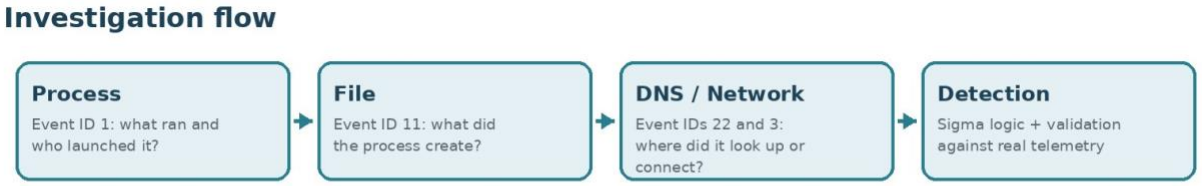


Figure 17. From telemetry generation to detection logic.

Validation boundary: The project performs two checks: YAML syntax/structure review and manual confirmation that the required fields appear in generated Sysmon telemetry.
It does not claim sigma-cli semantic validation, backend translation, SIEM rule execution or alert generation.

Step 26 - Create Sigma Rule 1 - PowerShell launching discovery tools

powershell-discovery.yml

```
title: PowerShell Launching Discovery Utilities
id: 609f59e9-dcb9-5e2e-8813-5a3893c9ee8c
status: experimental
description: Detects PowerShell launching whoami.exe or ipconfig.exe in the Project 2 lab.
author: Oluwaseyi Falode
date: 2026-09-28
logsource:
  product: windows
  category: process_creation
detection:
  selection_parent:
    ParentImage|endswith:
      - '\powershell.exe'
      - '\pwsh.exe'
  selection_child:
    Image|endswith:
      - '\whoami.exe'
      - '\ipconfig.exe'
  condition: selection_parent and selection_child
falsepositives:
  - Approved administration and troubleshooting
level: medium
tags:
  - attack.discovery
  - attack.t1033
  - attack.t1016
  - attack.t1059.001
```

Detection idea: PowerShell must be the parent and the child must be whoami.exe or ipconfig.exe. This is review-worthy discovery behavior, not automatic proof of compromise.

Step 27 - Create Sigma Rule 2 - suspicious script creation in Temp

Core detection logic

```
title: PowerShell Creating Script or Executable in User Temp
id: 72d913b4-da1f-5447-9cdc-ebb896547dbd
status: experimental
description: Detects PowerShell creating executable or script-like files in a user temporary directory.
author: Oluwaseyi Falode
date: 2026-09-28
logsource:
  product: windows
  category: file_event
detection:
  selection_process:
    Image|endswith:
      - '\powershell.exe'
      - '\pwsh.exe'
  selection_path:
    TargetFilename|contains:
      - '\Temp\'
      - '\AppData\Local\Temp\'
  selection_extension:
    TargetFilename|endswith:
      - '.exe'
      - '.dll'
      - '.ps1'
      - '.bat'
      - '.vbs'
  condition: selection_process and selection_path and selection_extension
falsepositives:
  - Approved script staging and software installation
level: medium
tags:
  - attack.execution
  - attack.t1059.001
```

Why not every Temp file?: Alerting on every Temp file would be noisy. The rule requires PowerShell, a user Temp path, and an executable/script-like extension.

Step 28 - Create Sigma Rule 3 - PowerShell outbound network connection

Core detection logic

```
title: PowerShell Outbound Network Connection
id: b69c7ac0-b8fb-5f6e-8201-c162b2e546a3
status: experimental
description: Detects outbound network connections initiated by PowerShell.
author: Oluwaseyi Falode
date: 2026-09-28
logsource:
  product: windows
  category: network_connection
detection:
  selection:
    Image|endswith:
      - '\powershell.exe'
      - '\pwsh.exe'
    Initiated: 'true'
  condition: selection
falsepositives:
  - Approved PowerShell web requests and management automation
level: medium
tags:
  - attack.execution
  - attack.t1059.001
```

False positives: PowerShell legitimately accesses the network for administration, package retrieval and automation. The rule is tagged only for PowerShell execution; the lab connection is not labeled command-and-control.

Step 29 - Validate Rule 1 against Sysmon Event ID 1

This step confirms that the positive Sysmon event contains the fields required by Rule 1. It does not execute the Sigma rule itself. Create a fresh time marker, run whoami, then search only Event ID 1 after that marker.

Positive-case validation

```
$t = Get-Date
whoami
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; StartTime=$t; Id=1} | Where-
Object {$_.Message -match 'whoami\.exe'} |
Select-Object -First 1 -ExpandProperty Message
```

```
PS C:\Users\windows\Downloads\Sysmon> $tcp = New-Object System.Net.Sockets.TcpClient
PS C:\Users\windows\Downloads\Sysmon> $tcp.Connect("1.1.1.1",443)
PS C:\Users\windows\Downloads\Sysmon> $tcp.Close()
PS C:\Users\windows\Downloads\Sysmon> $t = Get-Date
PS C:\Users\windows\Downloads\Sysmon> whoami
win-1nb187jrie5\windows
PS C:\Users\windows\Downloads\Sysmon> Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; StartTi
 -match 'whoami.exe'} | Select-Object -First 1 -ExpandProperty Message
Process Create:
RuleName: -
UtcTime: 2026-09-28 06:22:31.424
ProcessGuid: {b2aea449-07a7-6aba-5d07-000000000600}
ProcessId: 4996
Image: C:\Windows\System32\whoami.exe
FileVersion: 10.0.26100.1 (WinBuild.160101.0800)
Description: whoami - displays logged on user information
Product: Microsoft® Windows® Operating System
Company: Microsoft Corporation
OriginalFileName: whoami.exe
CommandLine: "C:\WINDOWS\system32\whoami.exe"
CurrentDirectory: C:\Users\windows\Downloads\Sysmon\
User: WIN-1NB187JRIE5\windows
```

Figure 20. Telemetry confirmation for Rule 1: fresh whoami.exe Event ID 1 with powershell.exe as ParentImage.

Result: Rule 1 required fields confirmed present in generated Sysmon telemetry.

Negative-condition check (manual): the earlier notepad.exe process event does not satisfy Rule 1 because the child Image is neither whoami.exe nor ipconfig.exe.

Step 30 - Validate Rule 2 against Sysmon Event ID 11

Create a harmless .ps1 file in Temp, then confirm that the resulting Event ID 11 contains the Image, path and extension fields required by Rule 2. This is telemetry-field confirmation, not Sigma-engine execution.

Positive-case validation

```
Set-Content "$env:TEMP\soc-test.ps1" -Value 'Write-Host "SOC Lab Test"'
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 200 |
Where-Object { $_.Id -eq 11 -and $_.Message -match "soc-test\.ps1" -and $_.Message -match "Image:.*powershell\.exe" } | Select-Object
-First 1 -ExpandProperty Message
```

```
>>     $_.Message -match "ParentImage:.*powershell.exe" -and
>>     $_.Message -match "Image:.*(whoami.exe|ipconfig.exe)"
>> } |
>> Select-Object -First 5 -ExpandProperty Message
PS C:\Users\windows\Downloads\Sysmon> whoami
win-1nb187jrie5\windows
PS C:\Users\windows\Downloads\Sysmon> Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 200 | Where-Obj
h "ParentImage:.*powershell.exe" -and $_.Message -match "Image:.*whoami.exe" } | Select-Object -First 1 -ExpandProperty Messa
PS C:\Users\windows\Downloads\Sysmon> Set-Content "$env:TEMP\soc-test.ps1" -Value 'Write-Host "SOC Lab Test"'
PS C:\Users\windows\Downloads\Sysmon> Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 200 | Where-Obj
```

Figure 21. Telemetry confirmation for Rule 2: PowerShell creates soc-test.ps1 and Event ID 11 records the target path.

Result: Rule 2 required fields confirmed present in generated Sysmon telemetry. The file was created for testing; its presence does not prove script execution.

Negative-condition check (manual): the earlier project2-file-evidence-2.txt Event ID 11 does not satisfy Rule 2 because .txt is not in the selected extension list.

Step 31 - Validate Rule 3 against Sysmon Event ID 3

Use the lab TCP test to confirm the fields required by Rule 3. Because the retrieved Event ID 3 record has an earlier UtcTime than the later validation screenshot, treat it as a separate network evidence record rather than attributing it solely by screenshot order.

Positive-case validation

```
$t = Get-Date
$tcp = New-Object System.Net.Sockets.TcpClient
$tcp.Connect('1.1.1.1',443)
$tcp.Close()
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; StartTime=$t; Id=3} |
Where-Object {$_.Message -match 'powershell\.exe'} |
Select-Object -First 1 -ExpandProperty Message
```

```
ParentProcessGuid: {b2aea449-bc9a-6ab8-6501-000000000600}
ParentProcessId: 1104
ParentImage: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
ParentCommandLine: "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
ParentUser: WIN-1NB187JRIE5\windows
PS C:\Users\windows\Downloads\Sysmon> $t = Get-Date
PS C:\Users\windows\Downloads\Sysmon> $tcp = New-Object System.Net.Sockets.TcpClient
PS C:\Users\windows\Downloads\Sysmon> $tcp.Connect("1.1.1.1",443)
PS C:\Users\windows\Downloads\Sysmon> $tcp.Close()
PS C:\Users\windows\Downloads\Sysmon> Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; StartTi
Network connection detected:Select-Object -First 1 -ExpandProperty Message
RuleName: -
UtcTime: 2026-09-27 16:44:45.436
ProcessGuid: {b2aea449-bc9a-6ab8-6501-000000000600}
ProcessId: 1104
Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
User: WIN-1NB187JRIE5\windows
Protocol: tcp
```

Figure 22. Network telemetry for Rule 3 field confirmation: powershell.exe initiates TCP to 1.1.1.1:443.

Result: Rule 3 required fields confirmed present in Sysmon telemetry. The destination is a test value, not an indicator of compromise.

Negative-condition check (manual): the earlier svchost.exe Event ID 3 network record does not satisfy Rule 3 because Image is not PowerShell.

Step 32 - Summarize detection coverage and limitations

Detection	Sysmon source	Confirmed fields	Status
PowerShell -> discovery utility	Event ID 1	Image + ParentImage	Telemetry fields confirmed
PowerShell -> script/executable in Temp	Event ID 11	Image + TargetFilename path/extension	Telemetry fields confirmed

PowerShell outbound connection	Event ID 3	Image + Initiated / destination context	Telemetry fields confirmed

What is not demonstrated: sigma-cli semantic validation, SIEM/backend translation, alert generation, full negative-rule execution, false-positive rate, precision/recall, 24/7 collection or enterprise response tooling.

Next professional step: run sigma check / sigma-cli with the chosen backend, translate each rule, replay positive and negative events, baseline expected PowerShell behavior and measure alert volume before promoting any rule beyond experimental.

Rule-file QA performed for this publication: all three YAML files parse successfully with PyYAML, use valid UUID identifiers, and contain the core Sigma sections (title, id, status, description, author, date, logsource, detection and level). sigma-cli was not available in the document-build environment, so Sigma semantic/backend validation is not claimed.

Step 33 - Write the incident response recommendation

Phase	Recommended action if this appeared unexpectedly in enterprise
Triage	Confirm the user, parent process, command line, host role and whether approved automation explains the activity.
Scope	Review neighboring Sysmon process/file/DNS/network events and identify additional affected users/hosts.
Preserve	Export logs and collect relevant files/hashes before cleanup when compromise is plausible.
Contain	Use approved EDR controls to isolate the endpoint or stop confirmed malicious activity when evidence supports it.
Eradicate	Remove confirmed malicious artifacts/persistence and remediate the established root cause.
Recover	Restore intended access, validate controls, then monitor for recurrence.
Improve	Tune detections based on the observed false-positive/truepositive context.

Proportionate response: Do not isolate a host or block an IP merely because PowerShell, Temp or port 443 appears. Response should follow evidence and organizational procedure.

Step 34 - Lessons learned and skills demonstrated

- Built and instrumented a Windows 11 ARM endpoint in UTM.
- Installed and configured Sysmon for process, file, network and DNS visibility.
- Queried Microsoft-Windows-Sysmon/Operational with Get-WinEvent and time-scoped filters.
- Reconstructed process lineage and explained parent/child relationships.
- Correlated ProcessGuid and PID fields across endpoint records.
- Investigated Event IDs 1, 3, 11 and 22.
- Distinguished raw telemetry from detection logic and analyst conclusions.
- Mapped supported behavior to MITRE ATT&CK T1033, T1016 and T1059.001.
- Wrote three Sigma detection rules, syntax-checked the YAML, and manually confirmed their required positive telemetry fields.
- Documented evidence limitations and avoided overstating malicious intent.

Portfolio takeaway: The project shows a complete SOC workflow: instrument -> generate -> observe -> scope -> correlate -> map -> detect -> validate -> recommend response.

Step A - Appendix A - Troubleshooting and common mistakes

Problem	Fix / lesson
VM lands in UEFI shell	Verify the ARM64 Windows ISO is attached and recreate the VM with the correct Windows virtualization path if needed.
Sysmon service name not found	On ARM64, verify Get-Service Sysmon64a rather than Sysmon64.
PowerShell cmdlets fail in C:\ prompt	You are inside cmd.exe. Exit back to a PS C:\...> prompt before running PowerShell cmdlets.
Event ID 11 returns nothing	Check the active Sysmon configuration. Install does not equal full event collection.
XML config cannot be opened	Verify sysmon-lab-config.xml exists in the Sysmon folder and is not accidentally saved as .xml.txt.
Long query returns no results	Use a time marker or larger time window rather than only -MaxEvents 100/200.
example.com request fails	Treat DNS failure separately; for network-rule validation, use a direct lab TCP connection and document the reason.
Two commands paste together	Run each pipeline separately. A valid command can fail if pasted directly after the previous command without a line break.

Rule: Do not replace a failed result with the expected output. Record what happened, understand why, then retest cleanly.

Step B - Appendix B - Evidence index, publishing notes and references

Figure	Evidence
2-4	UTM / Windows setup and Guest Tools
5	Sysmon install and service Running
6-8	Event ID 1 process-create and lineage/discovery
9-11	Telemetry gap, current config, successful config update
12	Event ID 11 file creation
13	Event ID 3 network telemetry
14	Event ID 22 DNS telemetry
15-16	Timeline and filtered incident table
18-19	Sigma YAML files
20-22	Detection validation evidence

Publishing checklist: crop unrelated desktop/UI areas, remove personal email/org names if they appear in future screenshots, keep only evidence that teaches something, and clearly label simulated activity as a lab.

Official references

Production comparison note: the lab config intentionally collects broadly for learning. For a production baseline, compare and tune against maintained community configurations such as SwiftOnSecurity/sysmon-config and Olaf Hartong/sysmon-modular; both recommend environment-specific tuning before broad deployment.

- Microsoft Sysmon - https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon
- Microsoft Get-WinEvent - https://learn.microsoft.com/powershell/module/microsoft.powershell.diagnostics/get-winevent
- MITRE ATT&CK T1033 - https://attack.mitre.org/techniques/T1033/
- MITRE ATT&CK T1016 - https://attack.mitre.org/techniques/T1016/
- MITRE ATT&CK T1059.001 - https://attack.mitre.org/techniques/T1059/001/
- Sigma rule basics - https://sigmahq.io/docs/basics/rules.html

Final conclusion: Project 2 demonstrates a reproducible endpoint-investigation workflow using Windows 11, Sysmon, PowerShell, MITRE ATT&CK and Sigma. The strongest evidence is the process lineage, file/network/DNS telemetry, timeline scoping and manual validation of three detection patterns.

Appendix C - Published configuration and corrected Sigma rules

These copy/paste-ready definitions reflect the reviewed public version. The lab originally accepted Sysmon schema 4.90; this appendix uses 4.91 to match the schema reported by Sysmon 15.22.

Sysmon lab configuration (published version)

```
<Sysmon schemaversion="4.91">
  <HashAlgorithms>sha256</HashAlgorithms>
  <EventFiltering>
    <ProcessCreate onmatch="exclude"></ProcessCreate>
    <NetworkConnect onmatch="exclude"></NetworkConnect>
    <FileCreate onmatch="exclude"></FileCreate>
    <DnsQuery onmatch="exclude"></DnsQuery>
  </EventFiltering>
</Sysmon>
```

Production comparison: https://github.com/SwiftOnSecurity/sysmon-config and https://github.com/olafhartong/sysmon-modular are stronger starting points for production-oriented tuning; this lab config is intentionally broad.

Rule 1 - powershell-discovery.yml

Corrected YAML

```
title: PowerShell Launching Discovery Utilities
id: 609f59e9-dcb9-5e2e-8813-5a3893c9ee8c
status: experimental
description: Detects PowerShell launching whoami.exe or ipconfig.exe in the Project 2 lab.
author: Oluwaseyi Falode
date: 2026-09-28
logsource:
  product: windows
  category: process_creation
detection:
  selection_parent:
    ParentImage|endswith:
      - '\powershell.exe'
      - '\pwsh.exe'
  selection_child:
    Image|endswith:
      - '\whoami.exe'
      - '\ipconfig.exe'
  condition: selection_parent and selection_child
falsepositives:
  - Approved administration and troubleshooting
level: medium
tags:
  - attack.discovery
  - attack.t1033
  - attack.t1016
  - attack.t1059.001
```

Publication QA: YAML syntax parsed successfully and required top-level Sigma fields were present. sigma-cli/backend translation was not executed in this environment, so no backend-specific query output is claimed.

PROJECT 02 / BUILD ALONG

Rule 2 - powershell-temp-file.yml

Corrected YAML

```
title: PowerShell Creating Script or Executable in User Temp
id: 72d913b4-da1f-5447-9cdc-ebb896547dbd
status: experimental
description: Detects PowerShell creating executable or script-like files in a user temporary directory.
author: Oluwaseyi Falode
date: 2026-09-28
logsource:
  product: windows
  category: file_event
detection:
  selection_process:
    Image|endswith:
      - '\powershell.exe'
      - '\pwsh.exe'
  selection_path:
    TargetFilename|contains:
      - '\Temp\'
      - '\AppData\Local\Temp\'
  selection_extension:
    TargetFilename|endswith:
      - '.exe'
      - '.dll'
      - '.ps1'
      - '.bat'
      - '.vbs'
  condition: selection_process and selection_path and selection_extension
falsepositives:
  - Approved script staging and software installation
level: medium
tags:
  - attack.execution
  - attack.t1059.001
```

Publication QA: YAML syntax parsed successfully and required top-level Sigma fields were present. sigma-cli/backend translation was not executed in this environment, so no backend-specific query output is claimed.

Rule 3 - powershell-network.yml

Corrected YAML

```
title: PowerShell Outbound Network Connection
id: b69c7ac0-b8fb-5f6e-8201-c162b2e546a3
status: experimental
description: Detects outbound network connections initiated by PowerShell.
author: Oluwaseyi Falode
date: 2026-09-28
logsource:
  product: windows
  category: network_connection
detection:
  selection:
    Image|endswith:
      - '\powershell.exe'
      - '\pwsh.exe'
    Initiated: 'true'
  condition: selection
falsepositives:
  - Approved PowerShell web requests and management automation
level: medium
tags:
  - attack.execution
  - attack.t1059.001
```

Publication QA: YAML syntax parsed successfully and required top-level Sigma fields were present. sigma-cli/backend translation was not executed in this environment, so no backend-specific query output is claimed.